Chapter 4 - Overview to RequestDispatcher and sendRedirect

4.1 Objectives

The objective of this topic is to

- describe the RequestDispatcher mechanism;
- write servlet code to create a request dispatcher;
- write servlet code to forward or include the target resource;
- identify and describe the additional request-scoped attributes provided by the container to the target resource.

4.2 Audience

This document is designed for Java EE Learners and programmers with a need to understand the Request Dispatcher Object and the sendRedirect method functionalities. The trainees who have understood the Servlets and its implementation are the intended audience.

4.3 Prerequisites

Associates should have gone through all the concepts of the Servlets and should have programmed Servlets for a Web Project.

4.4 Course Content

4.4.1 Introduction

A RequestDispatcher object can forward a client's request to a resource or include the resource itself in the response back to the client. A resource can be another servlet, or an HTML file, or a JSP file, etc.

You can also think of a RequestDispatcher object as a wrapper for the resource located at a given path that is supplied as an argument to the getRequestDispatcher method.

For constructing a RequestDispatcher object, you can use either

- ✓ **ServletRequest.getRequestDispatcher() method**
- ✓ **ServletContext.getRequestDispatcher() method**

They both do the same thing, but impose slightly different constraints on the argument path. For the former, it looks for the resource in the same webapp to which the invoking servlet belongs and the pathname specified can be relative to invoking servlet. For the latter, the pathname must begin with '/' and is interpreted relative to the root of the webapp.

To illustrate, suppose you want Servlet_A to invoke Servlet_B. If they are both in the same directory, you could accomplish this by incorporating the following code fragment in either the service method or the doGet method of Servlet_A:

```
RequestDispatcher dispatcher = getRequestDispatcher("Servlet_B");

dispatcher.forward( request, response );
```

where request, of type HttpServletRequest, is the first parameter of the enclosing service method (or the doGet method) and response, of type HttpServletResponse, the second. You could accomplish the same by

```
RequestDispatcher          dispatcher=getServletContext().getRequestDispatcher(
    "/servlet/Servlet_B" );

  dispatcher.forward( request, response );
```

4.4.2 RequestDispatcher Object and its methods

The RequestDispatcher interface provides **two** methods. They are:

RequestDispatcher interface provides two important methods

Methods	Description
void forward(ServletRequest request, ServletResponse response)	forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server
void include(ServletRequest request, ServletResponse response)	includes the content of a resource (servlet, JSP page, HTML file) in the response

Figure 1: Methods in a RequestDispatcher Object

How to get an object of RequestDispatcher:

getRequestDispatcher() method of ServletRequest returns the object of RequestDispatcher.

```
RequestDispatcher rs = request.getRequestDispatcher("hello.html");

rs.forward(request,response);
```

Figure 2: Forward in a RequestDispatcher Object

(or)

```
RequestDispatcher rs = request.getRequestDispatcher("hello.html");

rs.include(request,response);
```

Example

Figure 3: Include in a RequestDispatcher Object

In this example, we will show how RequestDispatcher is used to forward or include response of a resource in a servlet. Here we are using index.html to get username and password from the user, Validate servlet will validate the password entered by the user, if the user has entered "studytonight" as password, then he will be forwarded to Welcome servlet else the index.html will be shown again to the user.

Files to be created

1. index.html will have form field to get user information
2. Validate.java will validate the data entered by the user
3. Welcome.java will be the welcome page
4. web.xml

index.html

```
<form action="validate" method="post">

Name:<input type="text" name="user"/><br/>

Password:<input type="password" name="pass"/><br/>

<input type="submit" value="login"/>

</form>
```

Validate.java

```
import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

public class Validate extends HttpServlet {

public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String user=request.getParameter("user”);

        String pass=request.getParameter("pass");
```

```
        if(p.equals("servlet"){
        RequestDispatcher rd=request.getRequestDispatcher("servlet2");
        rd.forward(request, response);
        }
        else{
        out.print("Sorry UserName or Password Error!");
        RequestDispatcher
rd=request.getRequestDispatcher("/index.html");
        rd.include(request, response);
        }
    }
}
```

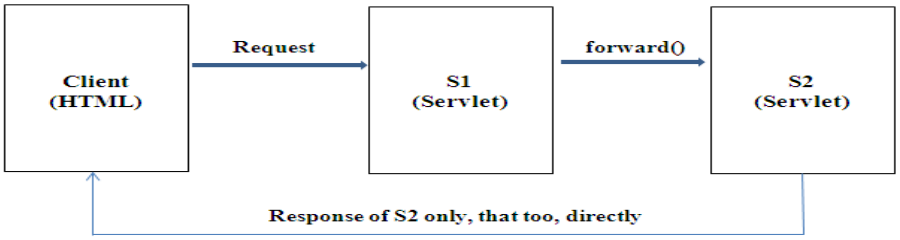
Welcome.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class Welcome extends HttpServlet {
        public  void  doPost(HttpServletRequest  request,  HttpServletResponse
```

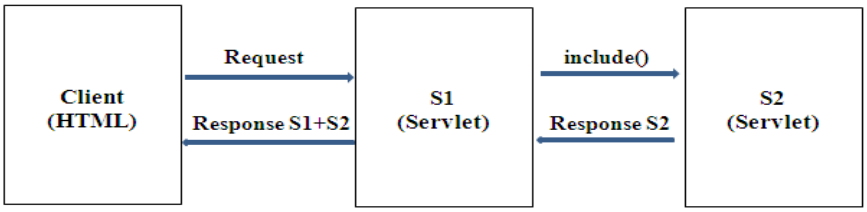
```
response)throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String n=request.getParameter("user");

        out.print("Welcome "+n);

        }

}
```

web.xml

```
<web-app>
 <servlet>
   <servlet-name>Validate</servlet-name>
   <servlet-class>Validate</servlet-class>
 </servlet>
 <servlet>
   <servlet-name>Welcome</servlet-name>
   <servlet-class>Welcome</servlet-class>
 </servlet>
 <servlet-mapping>
   <servlet-name>Validate</servlet-name>
   <url-pattern>/Validate</url-pattern>
 </servlet-mapping>
 <servlet-mapping>
   <servlet-name>Welcome</servlet-name>
   <url-pattern>/Welcome</url-pattern>
 </servlet-mapping>
 <welcome-file-list>
```



RequestDispatcher – forward() method



RequestDispatcher – include() method

```
  <welcome-file>index.html</welcome-file>

 </welcome-file-list>

</web-app>
```

Following figures give the visual difference you can grasp.

Figure 4: Forward and Include Controls in a RequestDispatcher

Property	include() Method	forward() Method
What can be done?	Includes another file in our current file. Will forward the	Will forward the client request to the forwarding

	client request to the forwarding page.	page.
Merge of response	Response of S1 and S2 are merged and sent to client (as if a single response). This way, the Programmer can achieve "server side includes".	No merge of response. Only S2 response will go to the client.
Retaining execution control	Shifted temporarily from S1 to S2. It works like a general simple method call.	Shifted permanently from S1 to S2
Control coming back	Execution control comes back to S1 after executing S2 for further processing of S1 after include() statement from where the execution control shifted.	Once shifted, the control never returns to S1. It is permanent shifting.
Response placement	Response of S2 is placed in S1.	Response of S2 is not placed in S2
Used when	Used when static information is to be included.	Used when dynamic information is to be included. It is can be used where a Servlet can play the role of a controller to process the client input

		and deciding what response page is to be returned.
Client response	Client receives the response from the same servlet which he requested.	Client actually receives the response from a different servlet (not known to client).
out.println	Output of S1′s out.println() statements go to client.	Output of S1′s out.println() statements never go to client.

Similarities:

- Both are methods of RequestDispatcher interface and are method calls.
- Both are abstract methods in the interface but can be treated as concrete methods in Servlet programming, because we are not implementing RequestDispatcher to our Servlet program, there is no compulsion to the programmer to override both.
- Processing done on the server when both methods are called is completely transparent (not visible or known) to client.
- Both are not treated as redirection.

4.4.3 sendRedirect() Method

response.sendRedirect() method

- It is a client side redirect.
- Sends a temporary redirect response to the client using the specified redirect location URL.
- When a sendRedirect method is invoked, it causes the web container to return to the browser indicating that a new URL should be requested. Because the browser issues a completely new request any object that are stored as request attributes before the redirect occurs will be lost.
- It is a method of HttpServletResponse interface.
- When a client sends a request for a particular page to a server and server sees that this request is can't be performed by this page, then it sends a error code to the browser specifying that the request can't be performed by this page. Along with the error code it also gives the address of the page which will be able to perform the request of the client.
- In sendRedirect() the object of request will be generated again with the location of page which will perform the request of the client.

The following code will explain you more about sendRedirect().

We are going to make one HTML in which we will submit the UserName and Password. The Controller Servlet will check if the password entered by the user is correct or not. If the password entered by the user is correct then the Servlet will redirect the request to the other Servlet which will handle the request. If the password entered by the user is wrong then it send error message to Browser.

Example :

Login.html

```
<html> <head> <title>New Page 1</title> </head>

<body>

<form method="POST" action="/SendRedirectServlet">

  <p>Enter UserName <input type="text" name="username" size="20"></p>

  <p>Enter Password <input type="hidden" name="password" size="20"></p>

  <p><input type="submit" value="Submit"></p>

</form>

</body> </html>
```

ControllerServlet .java

```
import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

public class SendRedirectServlet extends HttpServlet{

  protected void service(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {

    response.setContentType("text/html");

    PrintWriter out = response.getWriter();

    String name = request.getParameter("username");
```

```
  String password = request.getParameter("password");

  if(name.equals("vbrao")&& password.equals("JavaKen")){

    response.sendRedirect("/LoginServlet");

  }

  else{

    out.println("Your UserName or Password is incorrect");

  }  }}
```

LoginServlet .java

```
import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

public class LoginServlet extends HttpServlet{

protected void service(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {

  PrintWriter out = response.getWriter();

  out.println("Welcome to JavaKen.com ");}}
```

web.xml

```
<web-app>

 <servlet>
```

```
<servlet-name>controller</servlet-name>

<servlet-class>SendRedirectServlet</servlet-class>

</servlet>

<servlet-mapping>

<servlet-name>controller</servlet-name> <url-pattern>/SendRedirectServlet</url-
pattern>

</servlet-mapping>

<servlet>

<servlet-name>LoginSrv</servlet-name>

<servlet-class>LoginServlet</servlet-class>

</servlet>

<servlet-mapping>

<servlet-name>LoginSrv</servlet-name>

<url-pattern>/LoginServlet</url-pattern>

</servlet-mapping>

</web-app>
```

The following explains the above sample pictorially:

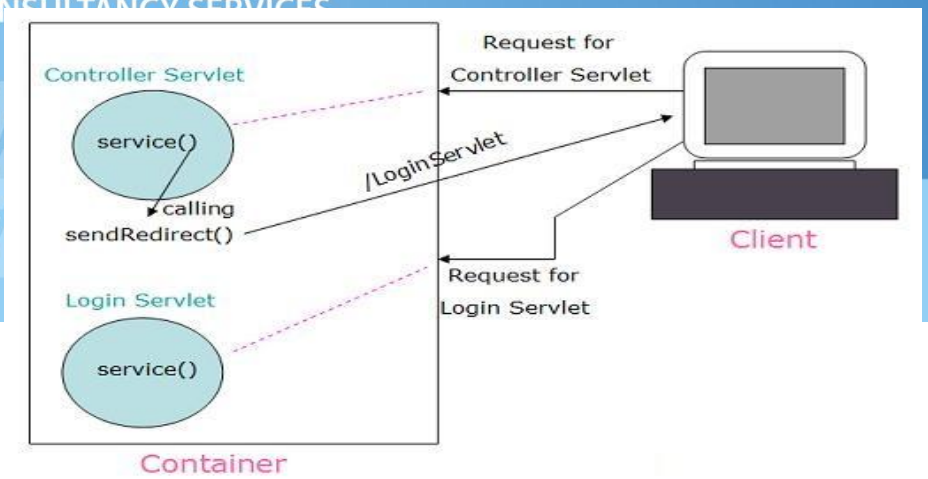


Figure 5: Controls in Sample Scenario

4.4.4 Difference - RequestDispatcher forward() and sendRedirect() methods

We have seen earlier include() and forward() methods of RequestDispatacher interface with example programs and differences were shown between include() and forward().

sendRedirect(String URL) is already discussed with example program.

Now we are going to discuss the difference between forward() and sendRedirect() methods.

Property	forward()	sendRedirect()
Defined interface	Defined in RequestDispatcher	Defined in HttpServletResponse
Signature	void forward(ServletRequest request, ServletResponse response)	void sendRedirect(String url)

Client awareness	Client is not aware of that he is getting response from a different Servlet as the URL will not change in client's browser.	Client can know easily as the URL (from where he is getting response) changes in the client browser's prompt.
Execution control	Execution control changes to another Servlet on the same server without client being informed that altogether a different Servlet is going to process his request.	Control changes to client
Where is what?	Forward is done on server side without client's knowledge.	Browser issues a new request on the URL that is redirected (sent as parameter) by the server and client can easily aware of.
Where happens	Everything happens on server side within the Web container and client is not involved.	sendRedirect() causes the Web container to return to the client's browser. Client intrun can redirect to different servers or domains.
Speed	Faster as forward runs on server-side entirely and no extra network trip is required.	Due to extra round trip between browser-server-browser (running on client as well as on server side), it is slower.
Content	forward() sends the same request to another resource of the same Web application.	Calls another page with a different request URL but not on the same request.

Usage	Used when processing is done by another Servlet	Used when wanted to redirect the client request to another Web site (completely out of current context) or to redirect the errors to another resource
Reusability	forward() reuses the current request object	Redirects create a new request object; consequently looses the original request with all its parameters and attributes.
Transfer of parameters	Original request and response objects transfer data coming from client along with additional information set with setAttribute() method (if any) to another resource request and response objects.	Redirect action sends header back to the client. Browser uses the URL contained in the header to call a new resource. As client initiates a new request, the original request and response objects are lost and fresh ones are to be created.
Transfer control	Internally, the Servlet container transfers control of client request to another servlet (or JSP).	This method sends the HTTP response to client browser to allow the client to send another request with a different URL. Usage of this method is equivalent to opening a new browser window and typing the URL.

What is sent?	Server sends the response (information required) to the client.	With this method, server sends a URL to the client.
Visual difference	Client cannot see the address of new resource that honours the client request in the address bar of the browser.	Client can see the new redirected address in address bar.
Examples	Calling another resource to process the data like validation of Login data.	Calling advertisements on the Web page or payment gateways.
Task separation	With this method, the responsibility of handling the client request can be distributed between many Servlets (or JSPs).	Used to transfer control altogether to a different domain. Also used to write separation of tasks.
Back and Forward buttons	As everything happens on server with forward, nothing is stored on browser history. So, Back and Forward buttons will not work.	As client makes new request and updated in browser history, back and forward buttons work.
URL	Use only relative URLs with forward().	Use absolute URLs.

Which one to prefer?	If you would like to forward the client request to a new resource on the same server for further process, prefer forward() where data of the original resource can be passed to the new resource.	If you would like to transfer the control to a new server or domain where client treats as a new task, prefer sendRedirect(). If the data of the original resource (which client requested) is needed in the new resource, store them in Session object and reuse.

4.4.5 How to communicate between two servlets in same applictaion

Communication between the Servlets is an important task to the programmer. Now let us see how to pass data between two servlets (one-to-one) and for this Servlet API comes with javax.servlet.RequestDispatcher interface.

In the following example code, client sends two numbers to a servlet to know their product. It calls a servlet GetMe (with alias name Servlet1 in web.xml). Servlet1 forwards the client request to Product servlet (of alias name Servlet2) using forward() method of RequestDispatcher interface.

TwoNumbers.html

```
<body>

 <h2>Product of Two Numbers</h2>

 <form method="get" action="Servlet1">

   Enter First Number   <input type="text" name="t1"> <br>

  Enter Second Number  <input type="text" name="t2"> <br>

  <input type="submit" value="PRODUCT PLEASE">

 </form>

</body>
```

Getme.java

```
import javax.servlet.*;

import javax.servlet.http.*;

import java.io.*;

public class GetMe extends HttpServlet

{  public void service(HttpServletRequest req, HttpServletResponse
res) throws ServletException, IOException

  {

    res.setContentType("text/html");
```

```
    PrintWriter out = res.getWriter();

     out.println("Hello Servlet1");      // will not go to client

      // create an object of RequestDispatcher

    RequestDispatcher rd = req.getRequestDispatcher("/Servlet2");   //
Servlet2 is the alias name of Product servlet

      // send the client data of two numbers available with req of Servlet1
to req of Servlet2 with forward()

    rd.forward(req, res);

     out.println("Hello Servlet2");      //  will not go to client

    out.close();

  }

}
```

Product.java

```
import javax.servlet.*;

import javax.servlet.http.*;

import java.io.*;

public class Product extends HttpServlet

          {
```

```
 public void service(HttpServletRequest req, HttpServletResponse res)
throws ServletException, IOException

  {    res.setContentType("text/html");

    PrintWriter out = res.getWriter();

   // extract the client data from req object.  Infact, this is actually to be
done by Servlet1

    String str1 = req.getParameter("t1");

    double num1 = Double.parseDouble(str1);

      // for second number, above two steps in one step

    double num2 = Double.parseDouble(req.getParameter("t2"));

      // now left is multiplication

    double result = num1 * num2;

        // prepare response.  This response will go to the client directly
ervlet1)

    out.println("Your first number: " + num1);

    out.println("<br>Your second number: " + num2);

    out.println("<br>Their product is " + result);

     out.close();

  }

}
```

Following is the **web.xml** entry for the two servlets.

```
<servlet>
  <servlet-name>Servlet1</servlet-name>
  <servlet-class>GetMe</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>Servlet1</servlet-name>
  <url-pattern>/Servlet1</url-pattern>
</servlet-mapping>
 <servlet>
  <servlet-name>Servlet2</servlet-name>
  <servlet-class>Product</servlet-class>
</servlet>
 <servlet-mapping>
  <servlet-name>Servlet2</servlet-name>
  <url-pattern>/Servlet2</url-pattern>
</servlet-mapping>
```

Now what client will receive and will not receive:

```
// response of Servlet1 (GetMe servlet) not received

    out.println("Hello Servlet1");

 // response of Servlet2 (Product servlet) will be received

   out.println("Your first number: " + num1);

   out.println("<br>"Your second number: " + num2);

   out.println("<br>Their product is " + result);



// response of Servlet1 (GetMe servlet) not received

     out.println("Hello Servlet2");
```

That is, response of Servlet1 never goes to the client and only Servlet2 goes (eventhough Hello Servlet1 exists before calling forward() method in GetMe servlet). Following figure explains

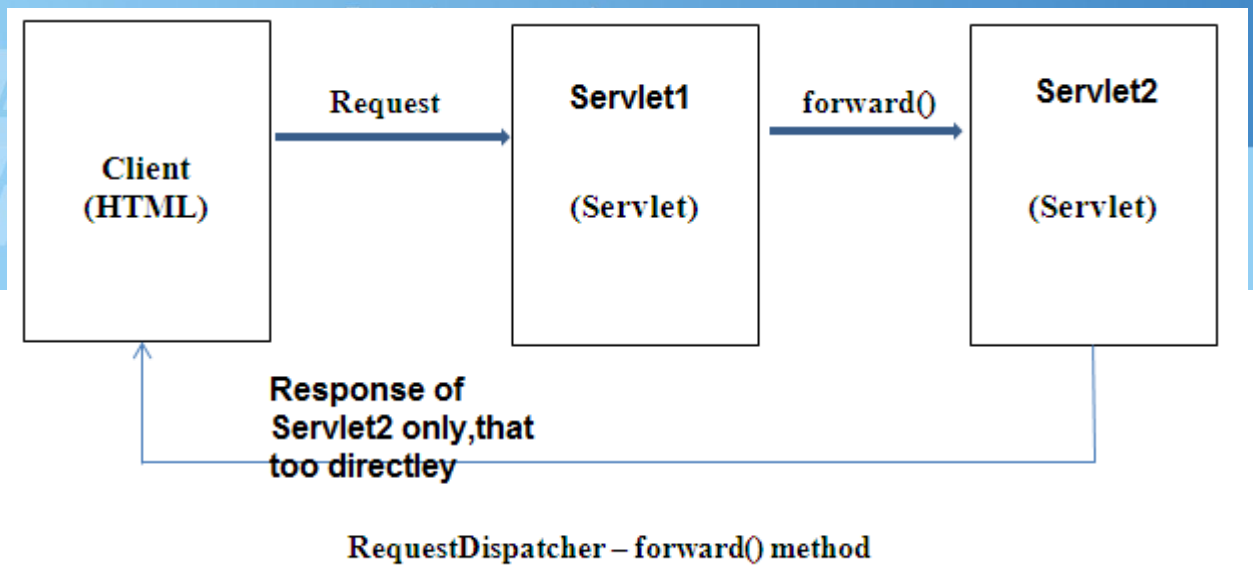


Figure 6: Control between two servlets

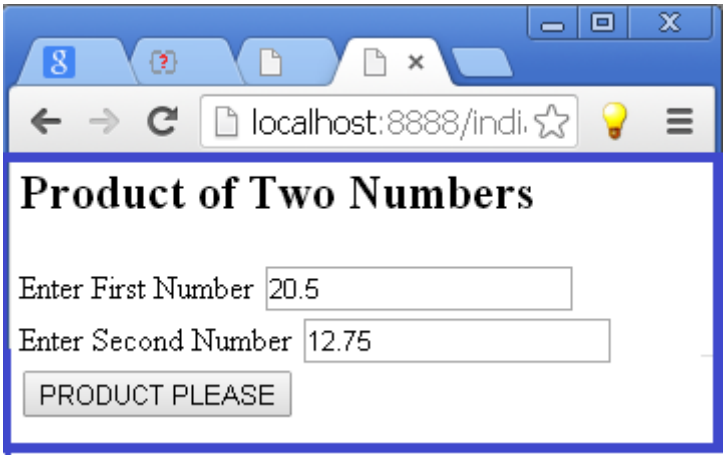
Observe the figure. Client request (or calls) the servlet Servlet1. Servlet1 sends client data to Servlet2 with forward() method. Response of S2 goes directly to client.

Let us see the screens.

Client Program: HTML file, TwoNumbers.html, when fields are filled up and clicked submit button.

Figure 7: Sample output -Control between two servlets

Observe the output. The output (response) of Servlet2 (Product servlet) only goes to client but not of Servlet1 (GetMe servlet). Hello Servlet1 and Hello Servlet2 of Servlet1 are not displayed in the



output screen.

4.4.6 How to communicate two servlets in same server with different application

Now let us see how two different servlet is going to communicate .Servlet "**SendRedirect**" is present in Application- "**WebApplicationOne"** and Servlet "PrintServler" is present Application "**WebApplicationTwo**".

Both the applications are present on the same server.

SendRedirect.java

```
package com.tcs.ilp;

import java.io.IOException;

import java.io.PrintWriter;

import javax.servlet.RequestDispatcher;

import javax.servlet.ServletConfig;

import javax.servlet.ServletContext;

import javax.servlet.ServletException;

import javax.servlet.http.HttpServlet;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;

/**
```

```
 * Servlet implementation class SendRedirect

 */

public class SendRedirect extends HttpServlet {

        private static final long serialVersionUID = 1L;

        /**

         * @see HttpServlet#doPost(HttpServletRequest request,
HttpServletResponse

         *      response)

         */

        protected void doPost(HttpServletRequest request,

                        HttpServletResponse response) throws ServletException,
IOException {

                ServletConfig config = getServletConfig();

                 ServletContext sc =
config.getServletContext().getContext("/WebApplicationTwo");

                 RequestDispatcher rd = sc.getRequestDispatcher("/PrintServlet");

                 rd.forward(request, response);

        }}
```

Web.xml (WebApplicationOne)

```
<servlet>
        <description></description>
        <display-name>SendRedirect</display-name>
        <servlet-name>SendRedirect</servlet-name>
        <servlet-class>com.tcs.ilp.SendRedirect</servlet-class>
        </servlet>
        <servlet-mapping>
        <servlet-name>SendRedirect</servlet-name>
        <url-pattern>/SendRedirect</url-pattern>
        </servlet-mapping>
```

PrintServlet.java

```
package com.tcs.ilp;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
```

```
/**
 * Servlet implementation class PrintServlet
 */
public class PrintServlet extends HttpServlet {
	private static final long serialVersionUID = 1L;
	/**
	 * @see HttpServlet#doPost(HttpServletRequest request,
HttpServletResponse response)	*/
	protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
		response.setContentType("text/html");
		PrintWriter out = response.getWriter();
		out.print("Application Two -- PrintServlet Called");
		out.flush();
	} }
```

web.xml (WebApplicationTwo)

```
<servlet>
	<description></description>
```

```
<display-name>SendRedirect</display-name>
<servlet-name>SendRedirect</servlet-name>
<servlet-class>com.tcs.ilp.SendRedirect</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>SendRedirect</servlet-name>
<url-pattern>/SendRedirect</url-pattern>
</servlet-mapping><servlet>
        <description>
        </description>
        <display-name>PrintServlet</display-name>
        <servlet-name>PrintServlet</servlet-name>
        <servlet-class>com.tcs.ilp.PrintServlet</servlet-class>
</servlet>
<servlet-mapping>
        <servlet-name>PrintServlet</servlet-name>
        <url-pattern>/PrintServlet</url-pattern>
</servlet-mapping>
```

Output:

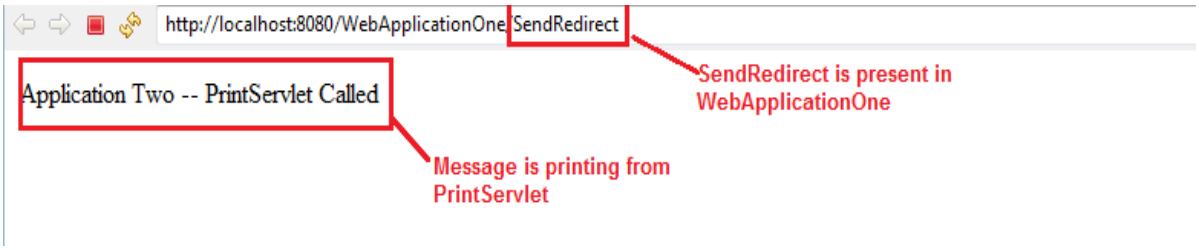


Figure 8: Sample Output with two web applications

4.4.7 Communication of two servlets in different servers in different application

Send Redirect can be used to communicate between two servlet present in different servers,the output will be same as request dispatcher forward example but the url of the page will be changed to redirected page

Servlet code:

res.sendRedirect("http://localhost:8086/Sample/Servlet2);

AuthenticateServlet.java which deployed in ApplicationServer1.

```
package com.tcs.ilp;



import java.io.*;

import java.io.IOException;

import javax.servlet.ServletException;

import javax.servlet.http.HttpServletRequest;
```

```
import javax.servlet.http.HttpServletResponse;

public class AuthendicateServlet extends javax.servlet.http.HttpServlet
implements javax.servlet.Servlet {

        private static final long serialVersionUID = 1L;

public AuthendicateServlet() {

super();

}

protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws

ServletException, IOException {

// TODO Auto-generated method stub

response.setContentType("text/html");

PrintWriter pw = response.getWriter();

String name = "tcs";

String password = "tcs";

if(name.equals("tcs")&& password.equals("tcs"))

{

//response.sendRedirect("/ServletProject/ValidUser");

        response.sendRedirect("url:http://localhost:8081/CheckServletProject/
GreetUserServlet");
```

```
}

else

{

pw.println("u r not a valid user");

}

}

}
```

web.xml

```
<servlet>

    <description></description>

    <display-name>AuthendicateServlet</display-name>

    <servlet-name>AuthendicateServlet</servlet-name>

    <servlet-class>com.tcs.ilp.AuthendicateServlet</servlet-class>

  </servlet>

  <servlet-mapping>

    <servlet-name>AuthendicateServlet</servlet-name>

    <url-pattern>/AuthendicateServlet</url-pattern>

  </servlet-mapping>
```

GreetUserServlet.java which is available at ApplicationServer2

```
package com.tcs.ilp;

import java.io.IOException;

import java.io.PrintWriter;

import javax.servlet.ServletException;

import javax.servlet.http.HttpServlet;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;

/**

 * Servlet implementation class GreetUserServlet

 */

public class GreetUserServlet extends HttpServlet {

        private static final long serialVersionUID = 1L;

  /**

    * @see HttpServlet#HttpServlet()

    */

   public GreetUserServlet() {

       super();

 }
```

```
	/**
 * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse response)
	 */
	protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
				response.setContentType("text/html");
				PrintWriter out = response.getWriter();
				out.print("CheckServletProject from Different Server Called");
				out.flush();
 }

	/**
	 * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse response)
	 */
	protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
			// TODO Auto-generated method stub
```

```
        }



}
```

web.xml

```
<servlet>
    <description></description>
    <display-name>GreetUserServlet</display-name>
    <servlet-name>GreetUserServlet</servlet-name>
    <servlet-class>com.tcs.ilp.GreetUserServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>GreetUserServlet</servlet-name>
    <url-pattern>/GreetUserServlet</url-pattern>
  </servlet-mapping>
```

Output:

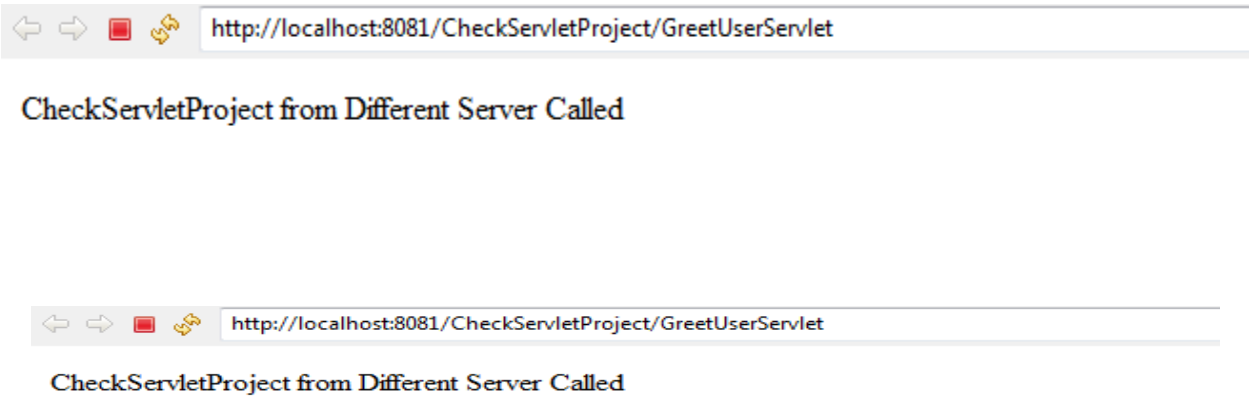


Figure 9: Sample Output with two web applications on different servers

4.4.8 Summary

In this topic, description about RequestDispatcher class was dealt. It was mainly used to 'pass on' the current request to another program (servlet) and therefore allows 'chaining' of the programs. You saw about RequestDispatcher that primarily contains two methods include() and forward(). include() method includes the response of another program while forward() method forwards the request of the current program to another one with relevant examples.